



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

COLLISION HANDLING TECHNIQUES

PRESENTED BY
SREE B(192524023)

INTRODUCTION

- Collision occurs when two or more records are assigned the same location in a hash table.
- Collision handling techniques help store and retrieve data efficiently.
- Efficient collision resolution techniques improve the performance of hash tables by minimizing search and insertion time.
- Common collision handling methods include Separate Chaining and Open Addressing, which ensure that all records can be stored even when collisions occur.

What is Collision?

A collision happens when different keys generate the same hash value.

- Example: $h(n) = k \bmod 10$
Key 101 $\rightarrow$ Address 1
Key 201 $\rightarrow$ Address 1
- Both records want to be stored at Address 1, causing a collision.

Why Handle Collisions?

- Prevent data loss.
- Improve search speed.
- Efficient memory usage

SEPARATE CHAINING

- Store multiple records in a linked list at the same hash location.

Advantages:

- Easy to implement.
- Can store many records.

Disadvantages:

- Extra memory required.
- Search time may increase.

Example : $k \% 10$ and $k=101,201,301,401$

Address 1 $\rightarrow$ 101 $\rightarrow$ 201 $\rightarrow$ 301 $\rightarrow$ 401

OPEN ADDRESSING

- Find another empty location when a collision occurs.
- Methods:
 1. Linear Probing $= h_i(k) = (h(k) + i) \bmod m$
 2. Quadratic Probing.
 3. Double Hashing.
- Advantages:
 - No extra memory needed.
- Disadvantages:
 - Performance decreases when table becomes full

LINEAR PROBING

- In **linear probing**, if a collision occurs, the next available slot is checked sequentially until an empty slot is found.
- The probing formula is:
- $h_i(k) = (h(k) + i) m$
- where:
- $h_i(k)$ = index checked on the i -th probe
- $h(k)$ = original hash value of key k
- i = probe number (0,1,2,3, ...)
- m = size of the hash table

Let's use the following:

- **Hash table size:** $m = 10$
- **Hash function:** $h(k) = k \bmod 10$
- **Keys:** 13, 23, 33, 43, 53

Since all these keys end in **3**, they all hash to index **3**, causing collisions.

Key	Hash Value $h(k)$	Probe Sequence	Final Position
13	$13 \bmod 10 = 3$	3	3
23	$23 \bmod 10 = 3$	$3 \rightarrow 4$	4
33	$33 \bmod 10 = 3$	$3 \rightarrow 4 \rightarrow 5$	5
43	$43 \bmod 10 = 3$	$3 \rightarrow 4 \rightarrow 5 \rightarrow 6$	6
53	$53 \bmod 10 = 3$	$3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$	7

Step-by-step insertion

1. Insert 13

1. $h(13) = 3$

2. Index 3 is empty $\rightarrow$ Place 13 at index 3

2. Insert 23

1. $h(23) = 3$

2. Index 3 occupied $\rightarrow$ Check 4

3. Index 4 empty $\rightarrow$ Place 23 at 4

3. Insert 33

1. $h(33) = 3$

2. 3 occupied $\rightarrow$ 4 occupied $\rightarrow$ 5 empty

3. Place 33 at 5

4. Insert 43

1. 3 occupied $\rightarrow$ 4 occupied $\rightarrow$ 5 occupied $\rightarrow$ 6 empty

2. Place 43 at 6

5. Insert 53

1. 3 occupied $\rightarrow$ 4 occupied $\rightarrow$ 5 occupied $\rightarrow$ 6 occupied $\rightarrow$
7 empty

2. Place 53 at 7

QUADRATIC PROBING

- In **quadratic probing**, if a collision occurs, the next position is found using the square of the probe number.
- $$h_i(k) = (h(k) + i^2) m$$
- where:
- $h_i(k)$ = index on the i -th probe
- $h(k)$ = original hash value
- $i = 0, 1, 2, 3, \dots$
- m = size of the hash table

Example

- **Hash table size:** $m = 10$
 - **Hash function:** $h(k) = k \% 10$
 - **Keys:** 13, 23, 33, 43, 53
- All keys hash to index 3.

Key	Hash Value	Probe Sequence	Final Position
13	3	3	3
23	3	3 $\rightarrow$ 4	4
33	3	3 $\rightarrow$ 4 $\rightarrow$ 7	7
43	3	3 $\rightarrow$ 4 $\rightarrow$ 7 $\rightarrow$ 2	2
53	3	3 $\rightarrow$ 4 $\rightarrow$ 7 $\rightarrow$ 2 $\rightarrow$ 9	9

Step-by-step

Insert 13

- $h(13) = 3$
- Index 3 is empty $\rightarrow$ Insert at **3**

Insert 23

- 3 occupied
- $(3 + 1^2) \% 10 = (3 + 1) = 4$
- Index 4 is empty $\rightarrow$ Insert at **4**

Insert 33

- 3 occupied
- 4 occupied
- $(3 + 2^2) \% 10 = (3 + 4) = 7$
- Index 7 is empty $\rightarrow$ Insert at **7**

Insert 43

- 3 occupied
- 4 occupied
- 7 occupied
- $(3 + 3^2) \% 10 = (3 + 9) = 12 \% 10 = 2$
- Index 2 is empty $\rightarrow$ Insert at **2**

Insert 53

- 3 occupied
- 4 occupied
- 7 occupied
- 2 occupied
- $(3 + 4^2) \% 10 = (3 + 16) = 19 \% 10 = 9$
- Index 9 is empty $\rightarrow$ Insert at **9**

CONCLUSION

Summary:

Collision occurs when two keys map to the same location.

Collision handling is important for efficient data storage.

Common techniques:

1. Separate Chaining.
2. Linear Probing.
3. Quadratic Probing.
4. Double Hashing.

THANK
YOU